ConveyorVision — Đề xuất dự án (nâng cấp sang YOLO26)

Cập nhật: Oct 11, 2026

1. Tóm tắt đề xuất

ConveyorVision chuyển từ pipeline "trích ROI cố định → trích đặc trưng thủ công (HOG) → classifier cổ điển (SVM/RF)" sang pipeline object detection end-to-end bằng **Ultralytics YOLO26** — model phát hành 14/1/2026, kiến trúc NMS-free, Dual-Head, tối ưu cho triển khai edge/biên với độ trễ ổn định.

Thay đổi cốt lõi: YOLO26 tự định vị (bounding box) **và** phân loại đạt/lỗi trong một lượt suy luận duy nhất trên mỗi frame, không cần bước ROI thủ công hay feature engineering riêng. Bài toán đếm sản phẩm trùng lặp giữa các frame — vốn là điểm cần chốt sớm của bản proposal gốc — được giải quyết trực tiếp bằng tracking tích hợp sẵn trong Ultralytics (ByteTrack), gán track ID cho từng sản phẩm xuyên suốt video.

Mục tiêu, phạm vi MVP và tên sản phẩm giữ nguyên như bản gốc; phần thay đổi tập trung ở baseline mô hình (mục 5), pipeline dữ liệu/suy luận (mục 3–4) và kiến trúc ứng dụng (mục 7).

2. Bài toán, giả định và tiêu chí thành công

Phát biểu bài toán (object detection)

Với mỗi frame I trích từ video, hệ thống dự đoán một tập bounding box {(b_i, c_i, s_i)}, trong đó b_i là toạ độ hộp, c_i ∈ {pass, defect} là lớp, và s_i là confidence score. Với confidence threshold t_conf và IoU threshold t_iou, hộp được giữ lại nếu s_i ⩾ t_conf; các hộp trùng lặp được lọc theo t_iou (YOLO26 NMS-free nên việc lọc trùng được xử lý ngay trong đầu ra của model, không cần bước NMS hậu xử lý riêng).

Giả định vận hành cần xác nhận

- Camera tương đối cố định; vẫn cần nhưng không còn bắt buộc ROI cố định — YOLO tự định vị sản phẩm trong khung hình.
- Có nhãn bounding box do người gán nhãn xác nhận cho từng sản phẩm xuất hiện trong frame (không chỉ nhãn ảnh/ROI như bản gốc).
- Mỗi sản phẩm vật lý được gán track ID xuyên suốt các frame nó xuất hiện; đếm sản phẩm dựa trên track ID, không dựa trên số frame.

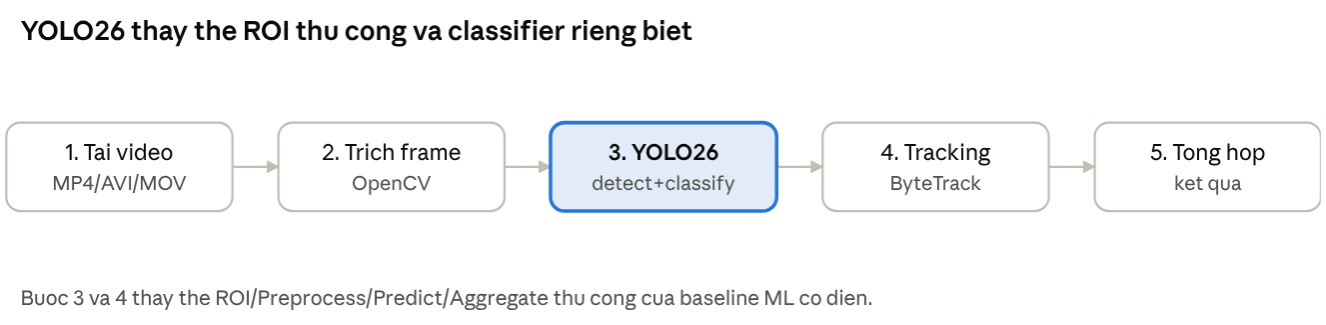
- Video có quyền sử dụng cho mục đích học tập/demo; không đưa dữ liệu nhạy cảm không cần thiết vào hệ thống.

Tiêu chí nghiệm thu đề xuất (định lượng sau pilot)

Nhóm	Tiêu chí	Cách xác nhận
Dữ liệu	Có manifest, nhãn bounding box dạng YOLO, ID video và quy tắc chia tập	Kiểm tra schema và audit mẫu nhãn
Model	So sánh baseline Majority Class với ít nhất 2 scale YOLO26 (vd. yolo26n, yolo26s)	Bảng mAP50/mAP50-95 trên cùng split
Lỗi bỏ sót	Báo cáo Recall lớp "lỗi" và số FN; chọn confidence threshold bằng validation	Confusion matrix cấp object + PR curve
Đếm sản phẩm	Không đếm trùng một sản phẩm qua nhiều frame	Audit track ID trên video mẫu
Ứng dụng	Upload → xử lý → trang kết quả có bounding box hoạt động trên demo	Test luồng thành công và file không hợp lệ
Tính trung thực	Không dùng test set để chọn mô hình/ngưỡng	Lưu cấu hình, seed, phiên bản dữ liệu và model

Không đặt trước điểm số giả: chưa có video/nhãn bounding box thực tế nên chưa thể báo cáo mAP, Recall hay tốc độ như kết quả đo; các con số chỉ được điền sau khi chạy thử nghiệm.

3. Pipeline tổng thể



pipeline ConveyorVision voi YOLO26 — 5 buoc

So với bản gốc (6 bước: Upload → Sampling → ROI → Preprocess → Predict → Aggregate), pipeline YOLO26 gộp ROI/Preprocess/Predict thành một bước inference duy nhất, và thêm bước tracking để chống đếm trùng sản phẩm.

Mô tả luồng

Bước	Đầu vào → đầu ra	Điểm kiểm soát
1. Upload	Video → file tạm + metadata	Kiểm tra đuôi/MIME, dung lượng, codec; đặt tên file an toàn
2. Sampling	Video → frame theo bước thời gian/FPS	Chọn sampling rate đủ bắt sản phẩm, tránh frame trùng gần nhau
3. YOLO26 inference	Frame → bounding box + class (pass/defect) + confidence	Dùng model.predict() với confidence/IoU threshold đã chốt từ validation
4. Tracking	Bounding box theo frame → track ID xuyên suốt video	Dùng tracker tích hợp (ByteTrack/BoT-SORT qua model.track())
5. Aggregate	Track ID + nhãn → summary / file kết quả	Đếm theo track ID, không đếm theo frame hay bounding box thô

Hai mức MVP (giữ nguyên tinh thần bản gốc, điều chỉnh theo YOLO)

Mức	Cách làm	Ưu / hạn chế
MVP-A: detect mỗi frame	Chạy YOLO26 trên từng frame độc lập, không tracking	Đơn giản, đủ cho demo; dễ đếm trùng nếu một sản phẩm xuất hiện nhiều frame
MVP-B: detect + track	Chạy model.track() để gán track ID, tổng hợp theo ID	Đếm chính xác số sản phẩm vật lý; cần kiểm tra chất lượng tracking khi sản phẩm che khuất nhau

4. Dữ liệu, gán nhãn và chống rò rỉ

Sự khác biệt lớn nhất so với bản gốc: YOLO cần nhãn **bounding box** cho từng sản phẩm trong frame, không phải nhãn ảnh/ROI đơn lẻ (0/1). Một frame có thể chứa nhiều bounding box nếu có nhiều sản phẩm.

Định dạng nhãn YOLO

Mỗi ảnh có một file `.txt` cùng tên, mỗi dòng là một đối tượng:

```
<class_id> <x_center> <y_center> <width> <height>
```

Toạ độ chuẩn hoá theo [0,1] trên kích thước ảnh; `class_id` = 0 (pass) hoặc 1 (defect).

Schema dữ liệu tối thiểu

Trường	Ví dụ/mô tả	Bắt buộc?
sample_id	Mã duy nhất của ảnh/frame	Có
video_id / lot_id	ID video, phiên quay hoặc lô sản xuất	Có, nếu có nguồn video
frame_idx / timestamp	Chỉ số frame hoặc thời gian trích xuất	Có
image_path	Đường dẫn ảnh gốc	Có
label_path	Đường dẫn file nhãn YOLO (.txt)	Có
bbox_count	Số bounding box trong ảnh	Khuyến nghị
annotator / review_status	Người gán nhãn, trạng thái kiểm tra	Khuyến nghị

Công cụ gán nhãn đề xuất

- **CVAT** hoặc **Roboflow**: hỗ trợ gán bounding box trực tiếp, xuất định dạng YOLO, có review kép.
- **LabelImg**: nhẹ, offline, phù hợp nhóm nhỏ.
- Cấu trúc thư mục chuẩn Ultralytics: `images/train`, `images/val`, `labels/train`, `labels/val`, cùng file `data.yaml` khai báo đường dẫn và tên lớp (`names: [pass, defect]`).

Quy tắc gán nhãn và chống rò rỉ (giữ nguyên nguyên tắc bản gốc)

- Viết checklist lỗi kèm ảnh ví dụ; thống nhất xử lý trường hợp mờ, che khuất hoặc không thể kết luận.
- Ưu tiên review kép một phần dữ liệu và ghi lại bất đồng.
- Vẽ bounding box sát viền sản phẩm; không bao trùm nhiều sản phẩm vào một box.
- **Chia Train/Validation/Test theo video/lot, không chia ngẫu nhiên từng frame** — vẫn là yêu cầu bắt buộc vì các frame liền kề gần như giống nhau, dù đơn vị nhãn đã đổi từ ảnh/ROI sang bounding box.

5. Thiết kế baseline — Ultralytics YOLO26

Thay vì tăng dần độ phức tạp qua nhiều họ mô hình (majority → logistic regression → HOG+SVM → random forest), baseline mới tăng dần theo **scale của cùng một kiến trúc YOLO26**, giúp so sánh công bằng giữa tốc độ và độ chính xác mà không đổi thuật toán.

Điểm kiến trúc đáng chú ý của YOLO26 (so với pipeline cổ điển)

- **NMS-free, end-to-end**: loại bỏ bước Non-Max Suppression hậu xử lý, giúp độ trễ ổn định hơn và dễ export sang ONNX/TensorRT.
- **Dual-Head**: trong huấn luyện dùng kiến trúc one-to-many hỗ trợ học tốt hơn; khi suy luận chuyển sang one-to-one cho đầu ra trực tiếp, không cần giải mã phức tạp (Distribution Focal Loss như các bản YOLO trước).
- **Hướng tới edge**: tối ưu cho triển khai trên phần cứng biên (Jetson, CPU), phù hợp với mục tiêu demo web/latency thấp của ConveyorVision.
- **Đa tác vụ**: cùng framework hỗ trợ detection, segmentation, pose, classification — nếu sau này cần phân loại nhiều *loại lỗi* chi tiết hơn nhị phân đạt/lỗi, có thể mở rộng mà không đổi stack.

Bảng baseline đề xuất

ID	Baseline	Scale	Ghi chú
B0	Majority Class	—	Mốc kiểm tra tối thiểu; Recall lớp lỗi = 0
B1	YOLO26n (nano)	~nhẹ nhất	Pretrained COCO, fine-tune; nhanh, phù hợp demo/CPU
B2	YOLO26s (small)	trung bình	Cân bằng tốc độ/độ chính xác; baseline chính đề xuất
B3	YOLO26m (medium)	lớn hơn	Dùng nếu B1/B2 chưa đạt Recall mục tiêu và có GPU huấn luyện

Pipeline huấn luyện

- Transfer learning từ checkpoint pretrained trên COCO (`yolo26n.pt`, `yolo26s.pt`...), fine-tune trên dữ liệu sản phẩm đạt/lỗi.
- Augmentation tích hợp sẵn trong Ultralytics (mosaic, flip, color jitter) — không cần tự viết pipeline augmentation riêng như bản ML cổ điển.
- Dò hyperparameter cơ bản (epochs, imgsz, batch) bằng validation; chia nhóm theo video/lot như mục 4.

- Lưu lại: file `.pt` đã fine-tune, `data.yaml`, confidence/IoU threshold đã chọn, phiên bản dữ liệu.

Code mẫu khởi động

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")  # pretrained COCO
model.train(data="data.yaml", epochs=100, imgsz=640, batch=16)

results = model.predict("frame.jpg", conf=0.35, iou=0.5)
```

Lưu ý về scale: bắt đầu từ yolo26n để có baseline nhanh và rẻ; chỉ chuyển lên yolo26s/m nếu mAP hoặc Recall lớp lỗi chưa đạt yêu cầu sau khi đã thử tăng dữ liệu/epochs.

6. Đánh giá mô hình và chọn ngưỡng

Bộ chỉ số chuyển từ cấp ảnh/ROI (bản gốc) sang cấp **object** (mỗi bounding box là một đơn vị đánh giá), thêm chỉ số chuẩn của object detection.

Chỉ số	Ý nghĩa	Vai trò
mAP50	Mean Average Precision tại IoU=0.5	Chỉ số tổng hợp chuẩn của object detection
mAP50-95	mAP trung bình trên nhiều ngưỡng IoU (0.5–0.95)	Đánh giá độ chính xác định vị box chặt chẽ hơn
Recall lớp lỗi	TP / (TP + FN) theo object	Ưu tiên cao nhất — như bản gốc, không đổi
Precision lớp lỗi	TP / (TP + FP) theo object	Kiểm soát báo động nhầm
Confusion matrix cấp object	So khớp box dự đoán với box ground-truth theo IoU	Thay thế confusion matrix cấp ảnh
Latency	Thời gian inference mỗi frame (ms)	Đo trên môi trường triển khai đã ghi cấu hình

Quy trình chọn ngưỡng (có 2 ngưỡng thay vì 1)

1. Huấn luyện trên Train; lấy prediction trên Validation.
2. Quét **confidence threshold**: ngưỡng tin cậy để giữ lại một dự đoán — ảnh hưởng trực tiếp Recall/Precision như threshold xác suất ở bản gốc.

3. Quét **IoU threshold**: ngưỡng chồng lấp để xác định một box là TP hay FP so với ground-truth (thường cố định 0.5 cho mAP50, có thể điều chỉnh theo yêu cầu định vị chặt/lỏng).
4. Chọn confidence threshold theo yêu cầu nghiệp vụ về Recall và chi phí FP/FN đã thống nhất — không mặc định dùng giá trị mặc định của thư viện.
5. Khoá model + 2 ngưỡng; chỉ dùng Test một lần cho báo cáo cuối.

7. Kiến trúc ứng dụng và API

Kiến trúc tổng thể giữ nguyên (web → FastAPI → video worker → model → kết quả JSON); thay đổi nằm ở lớp ML và một phần Video/CV.

Lớp	Công nghệ	Thay đổi so với bản gốc
Frontend	HTML/CSS/JS hoặc React	Không đổi; thêm hiển thị bounding box trên ảnh/video
API	FastAPI	Không đổi
Video/CV	OpenCV, NumPy	Bỏ bước trích HOG thủ công — YOLO26 tự học đặc trưng
ML	**Ultralytics (YOLO26)**	Thay cho scikit-learn + joblib; inference qua `model.predict()` / `model.track()`
Lưu trữ demo	Thư mục riêng + SQLite	Không đổi

API tối thiểu (không đổi so với bản gốc)

Method/path	Mục đích	Response rút gọn
POST /api/videos	Upload video, tạo job	`{"job_id": "...", "status": "queued"}`
GET /api/jobs/{job_id}	Tra trạng thái xử lý	`{"status": "processing|done|failed"}`
GET /api/jobs/{job_id}/results	Lấy tổng hợp kết quả	summary, predictions (bbox + track_id), metrics nếu có nhãn thật
GET /api/health	Kiểm tra dịch vụ	`{"status": "ok"}`

Thiết kế an toàn tối thiểu giữ nguyên như bản gốc: giới hạn dung lượng/thời lượng, whitelist định dạng, không tin tên file do client gửi, timeout, lưu file ngoài thư mục public, xoá file tạm theo TTL.

8. Giao diện demo và định dạng kết quả

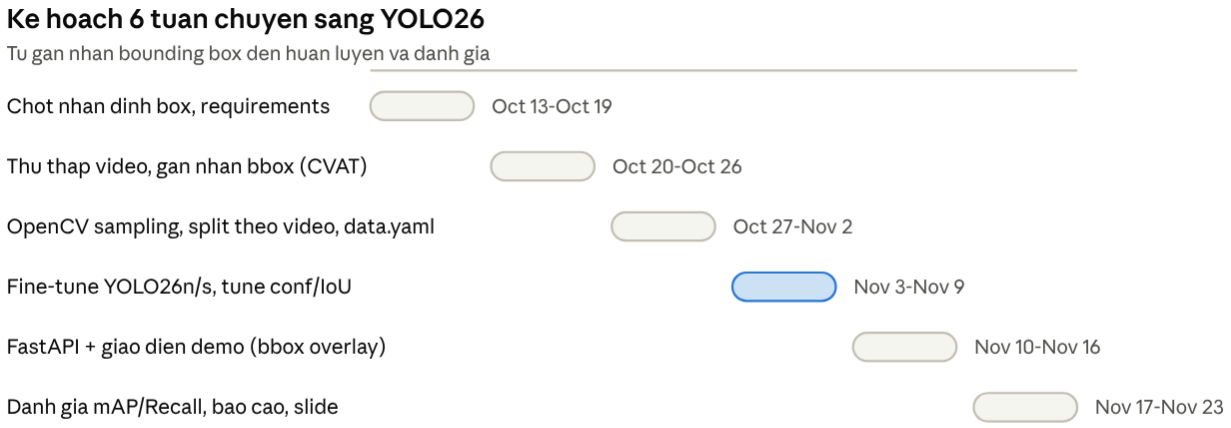
Nội dung cần hiển thị

- Trạng thái job: đang chờ/đang xử lý/hoàn tất/thất bại kèm lý do dễ hiểu (không đổi).
- **Bounding box vẽ trực tiếp lên frame/video**, kèm nhãn lớp (pass/defect) và confidence score — thay cho chỉ hiển thị nhãn ảnh như bản gốc.
- Tóm tắt số sản phẩm đã phát hiện theo track ID (không phải theo số frame), số dự đoán đạt, số dự đoán lỗi.
- Danh sách kết quả kèm timestamp/frame index, track_id, class, confidence, toạ độ bounding box, thumbnail.
- Biểu đồ phân bố confidence, confusion matrix cấp object khi người dùng cung cấp ground truth, mAP/Recall/Precision khi đủ nhãn.
- Tải CSV/JSON; phân biệt rõ "số bounding box đã phát hiện" với "số sản phẩm vật lý" (đã khử trùng theo track ID).

Ví dụ JSON (minh hoạ cấu trúc)

```
{
  "job_id": "demo-001",
  "status": "done",
  "analysis_unit": "bbox",
  "confidence_threshold": 0.35,
  "iou_threshold": 0.5,
  "summary": {"tracked_products": 42, "predicted_defect": 3,
"predicted_pass": 39},
  "predictions": [
    {"frame_idx": 42, "timestamp_s": 1.40, "track_id": 7, "label": "defect",
"confidence": 0.82, "bbox": [0.41, 0.52, 0.18, 0.22]}
  ],
  "note": "summary.tracked_products deduplicated by track_id; predictions
list is per detected bbox per frame."
}
```

9. Kế hoạch thực hiện và deliverables



ke hoach 6 tuan chuyen sang YOLO26

Tuần	Công việc chính	Deliverable
1	Chốt định nghĩa lỗi, quy tắc vẽ bounding box, cấu trúc repo	Data card, checklist gán nhãn bbox
2	Thu thập video, gán nhãn bounding box (CVAT/Roboflow), audit chất lượng	Dataset v0.1 dạng YOLO + data.yaml
3	OpenCV sampling, split theo video/lot, dựng data.yaml	Script tiền xử lý + cấu trúc dataset
4	Fine-tune yolo26n/yolo26s, dò confidence/IoU threshold	Model artifact (.pt) + bảng mAP/Recall
5	FastAPI, worker dùng model.predict()/track(), giao diện demo có bounding box overlay	Web demo end-to-end
6	Kiểm thử, đo latency, hoàn thiện báo cáo và slide	Final report, slide, README, model card

10. Rủi ro, giới hạn và cách giảm thiểu

Rủi ro	Tác động	Biện pháp giảm thiểu
Gán nhãn bounding box tốn công hơn nhãn ảnh	Chi phí/thời gian chuẩn bị dữ liệu tăng so với bản ML cổ điển	Dùng CVAT/Roboflow hỗ trợ auto-label gợi ý, review kép một phần

Rủi ro	Tác động	Biện pháp giảm thiểu
Cần GPU để huấn luyện hiệu quả	Train trên CPU rất chậm với ảnh độ phân giải cao	Bắt đầu từ yolo26n, ảnh nhỏ (imgsz=640 hoặc thấp hơn); dùng GPU free-tier (Colab/Kaggle) cho pilot
Overfitting với dataset nhỏ	mAP cao trên validation nhưng kém ngoài thực tế	Augmentation tích hợp sẵn, transfer learning từ COCO, theo dõi mAP50-95 không chỉ mAP50
Tracking lỗi khi sản phẩm che khuất nhau	Đếm sai số sản phẩm vật lý (trùng hoặc mất track ID)	Audit mẫu video có sản phẩm dày đặc/chồng lấp; cân nhắc tốc độ băng chuyền khi chọn FPS sampling
Leakage giữa frame	Metric validation/test cao giả tạo	Chia theo video/lot — không đổi so với bản gốc
License Ultralytics (AGPL-3.0/Enterprise)	Ràng buộc khi dùng thương mại/đóng mã nguồn	Xác nhận phạm vi sử dụng (học thuật/demo thường ổn với AGPL-3.0); cân nhắc Enterprise license nếu triển khai thương mại
Video lớn/codec lỗi	Timeout, treo worker	Giới hạn file, timeout, status job — không đổi so với bản gốc

Giới hạn cần nêu khi bảo vệ

- YOLO26 giải quyết tốt bài toán định vị + phân loại trong một bước, nhưng vẫn cần dữ liệu gán nhãn bounding box đủ đa dạng về góc camera, ánh sáng và mật độ sản phẩm.
- Kết quả mAP/Recall chỉ có ý nghĩa với dữ liệu/camera/điều kiện đã kiểm tra, không tuyên bố tổng quát hoá khi chưa test theo ca/lô độc lập.
- Tracking (ByteTrack) giảm đếm trùng nhưng không tuyệt đối chính xác khi sản phẩm di chuyển nhanh hoặc che khuất kéo dài — cần audit mẫu trước khi dùng số liệu đếm làm báo cáo chính thức.
- Hệ thống hỗ trợ quyết định kiểm tra; không điều khiển cơ cấu loại sản phẩm tự động trong MVP (giữ nguyên như bản gốc).

11. Phụ lục — pseudo-code và tài liệu tham khảo

Pseudo-code pipeline huấn luyện và đánh giá

```
1. Load dataset.yaml (images/labels train/val/test), groups = video_id/lot_id
2. Split groups -> Train / Validation / Test theo video/lot; khoá Test
3. model = YOLO("yolo26n.pt")  # pretrained COCO
4. model.train(data="data.yaml", epochs=.., imgsz=.., batch=..)
5. preds = model.predict(val_images, conf=sweep_values, iou=0.5)
6. Sweep confidence threshold -> chọn theo Recall/Precision/FN trade-off
7. Khoá model + 2 ngưỡng (confidence, IoU)
8. Đánh giá một lần trên Test: mAP50, mAP50-95, Recall/Precision lớp lõi,
   confusion matrix cấp object, FN/FP, latency
9. Lưu model .pt (+ export ONNX/TensorRT nếu cần), threshold, config, phiên
bản dữ liệu
```

Những quyết định chưa thể chốt nếu chưa có data

- Kích thước ảnh (imgsz) và sampling rate phù hợp tốc độ băng chuyền.
- Confidence threshold và IoU threshold mục tiêu, cùng chi phí FP/FN chấp nhận được.
- Scale model (yolo26n/s/m) phù hợp với phần cứng triển khai thực tế.
- Số lượng ảnh/bounding box cần thu thập dựa trên pilot và độ hiếm của lớp lõi.

Tài liệu tham khảo kỹ thuật

- Ultralytics YOLO26 — Official Docs
- Ultralytics — Model Training (train)
- Ultralytics — Object Tracking (ByteTrack/BoT-SORT)
- FastAPI — Request files/UploadFile
- OpenCV — Video I/O (VideoCapture)

Thông điệp khi trình bày: điểm mạnh không phải hứa trước mAP cao, mà là pipeline object detection end-to-end gọn hơn bản ML cổ điển, giải quyết trực tiếp vấn đề đếm sản phẩm trùng lặp qua tracking, và vẫn giữ kỷ luật chia dữ liệu/threshold như bản proposal gốc.